Gates Pass, Code Fails: A Measurement-Validity Study of LLM-as-Judge Test Adequacy for Requirement-to-Test Generation

Poojitha J, Arushi Tandon, Brahma Reddy Korraprolu, Y. Raghu Reddy
Software Engineering Research Center
International Institute of Information Technology, Hyderabad
Hyderabad, India

***Abstract*—Large language models (LLMs) are increasingly used to turn natural-language software requirements directly into test cases, yet evaluating the result remains unsolved. Text-similarity metrics (BLEU, ROUGE, CodeBLEU) are widely recognized as inadequate; the emerging alternative hands generated tests to a second LLM and asks it to judge correctness. This paper asks how much that judge should be trusted. We construct a three-gate evaluation pipeline (representational suitability (RSS), syntactic validity (SFV), and LLM-judged semantic adequacy with groundedness (FSA+M_g)) and use it to benchmark 10 open-weight LLMs generating tests from three real software requirement specifications (496 test suites, 4,058 test cases). The benchmark surfaces useful comparative findings: model size does not predict adequacy (Spearman $\rho = -0.015$, $p = 0.80$, $n = 311$), code-specialized models do not outperform general-purpose ones (Mann-Whitney $p = 0.14$), and test-case representation choice has a larger and more consistent effect on adequacy than model identity. Our primary contribution, however, is a measurement-validity audit of the pipeline itself: when we check gate verdicts against real test execution, the LLM-based audit agrees with execution on only 2 of 11 test cases (18%; 95% Wilson CI: 5%–48%); only 2 of 315 generated tests actually pass on execution for one case study; and 78% of judged suites (243 of 311) assert at least one concrete value not traceable to the source requirement, including 168 of the 214 suites that cleared the adequacy gate. We conclude that gate-passing, as currently operationalized, is a measurably weak proxy for correctness, and that any LLM-as-judge pipeline needs an explicit, reported, execution-grounded validity check.**

***Index Terms*—test case generation, large language models, requirements-based testing, LLM-as-judge, test adequacy evaluation, software testing, test representation**

I. Introduction

Software requirements specifications (SRS) describe what a system must do in natural language [1]; test cases verify that it does [2]. Turning the first into the second has always been labour-intensive [5], [6], and large language models (LLMs) are now routinely proposed to automate it, generating unit tests, behavioural scenarios, or formal test obligations directly from a requirement's text [7]–[9]. The question this raises is not whether LLMs can produce *something* that looks like a test case; they reliably can. The question is whether what they produce is the *right* test case for that requirement, and whether we can tell.

Text-similarity metrics such as BLEU [10], ROUGE [11], and CodeBLEU [12] answer a different question than the one that matters: they measure how closely generated text resembles a reference, not whether the generated test exercises the correct behaviour, uses an appropriate representation for the kind of requirement it came from, or asserts values the requirement actually specifies. A test case can score well on n-gram overlap while testing nothing the requirement cares about, and score poorly while being a perfectly adequate test phrased differently from the reference. This is well understood in the LLM-for-testing literature [13]–[18]. The natural response has been to stop comparing generated-text wording to reference-text wording, and instead hand the generated test to a second LLM that reads it and judges whether it is actually correct [19]–[21].

That response trades one hard-to-trust proxy for another. An LLM judge can be wrong in ways that are harder to detect than a low BLEU score: it can be confidently, fluently wrong [22], [23]. This paper's contribution is not another judge, but a direct measurement of how much a reasonably careful judge-based pipeline should be trusted [24], [25], obtained by building such a pipeline, running it at benchmark scale, and then checking its verdicts against the one thing that is not a proxy: whether the generated tests actually execute against, and agree with, the real implementation.

Research Questions. We organize our investigation around four RQs:

RQ1 Does LLM model size or code specialization predict test-suite adequacy as judged by an LLM-as-judge pipeline?
RQ2 Does test-case representation choice affect LLM-judged adequacy more than model identity?
RQ3 How well do LLM-judge gate-pass rates predict real test execution success against the implementation?
RQ4 To what extent do generated test suites assert concrete values not grounded in the source requirement (hallucination), and can that be detected by the gate pipeline?

Contributions.

1) A three-gate evaluation framework, comprising Representational Suitability (RSS), Syntactic Form Validity (SFV), and Functional Semantic Adequacy with Groundedness (FSA+M_g), with closed-loop repair, grounded in a 30-representation taxonomy of test-case forms (Sections III to IV).

2) A benchmark of 10 open-weight LLMs generating tests from three real SRS case studies: 496 test suites, 4,058 test cases, 29 project×model pairs, each suite also compared against the real implementation (Sections V to VI).
3) A quantified measurement-validity gap between what the gates report and what execution against real code shows: three independent checks (audit agreement, execution pass rate, and representation-choice agreement with an independent heuristic) all land far below what the gate-pass rates alone would suggest (Section VII).
4) A recommendation, supported by that gap, that groundedness be reported and thresholded as its own first-class check rather than folded into a single blended adequacy score.

II. Motivation: Why Not Just Score Similarity?

To establish that the problem this paper studies is not a straw man, we conducted a systematic review of the literature on LLMs applied to software testing [26]. Starting from 1,200 preprint sources (63 discarded for having zero citations) merged with published-venue results, we categorized 1,392 papers into four relevance tiers: 127 core, 501 relevant, 484 tangential, and 280 excluded (709 were preprints, 683 published). Of these, 1,315 were independently screened for inclusion by two of the authors, with a third author arbitrating disagreements. Inter-rater agreement was moderate (unweighted Cohen's [27] $\kappa = 0.5253$, 84.79% observed agreement, $p < 0.0001$, 95% CI $[0.4702, 0.5805]$) and substantial once near-misses ("maybe" judgments) are weighted as partial agreement (linear-weighted $\kappa = 0.6187$, quadratic-weighted $\kappa = 0.6790$). The final screened portfolio retained 183 papers (13.9%).

Inter-rater agreement of $\kappa = 0.5253$ is moderate by Landis & Koch's scale [28] and acceptable for an exploratory SLR in a rapidly evolving field; the linear-weighted $\kappa = 0.6790$ reflects that most disagreements were between `include` and `maybe` rather than between `include` and `exclude`. The third-author arbitration resolved 196 disputed papers, 84 of which changed outcome (52 additional inclusions, 32 additional exclusions). This process underscores a finding also visible in the retained portfolio. The underlying literature is similarly inconsistent about what evaluating an LLM-generated test should mean: surface similarity, compilation, line coverage, and human judgment are all in active use within the same subfield, rarely cross-validated against one another [7], [14]–[16]. This motivates treating evaluation methodology itself, rather than any particular generation technique, as the object of study.

III. A Taxonomy of Test-Case Representations

A requirement's natural test representation depends on what kind of behaviour it describes [29], [30]: a state-dependent requirement is naturally tested with a finite state machine or a transition table [31], [32]; a concurrency requirement with an interaction diagram or a temporal-logic obligation [33], [34]; a pure business rule with a decision table [35]. Treating *generate a test* as representation-agnostic discards this structure. To ground Gate 1 (Section IV) in something more principled than intuition, we conducted a second, separate systematic review targeted specifically at the test-case representation literature: 412 candidate papers (185 IEEE Xplore, 112 ACM Digital Library, 115 Google Scholar) were deduplicated to 328, reduced to 95 after title/abstract screening, and reduced to 30 after full-text review, each of the 30 retained papers anchoring one distinct test-case representation.

TABLE I
The six representation paradigms (30 representations total)

Paradigm	Representative forms
Scenario/Behavioral	Gherkin (BDD) [36], use cases [37], user stories
Stateful/Transitions	finite state machines [31], statecharts [38]
Concurrency/Interaction	sequence diagrams [33], GUI event graphs [39], [40]
Logical/Combinatorial	[41]–[43] decision tables [35], test requirement matrices
Programmatic/Code-level	xUnit test cases [44], UML Testing Profile [45]
Formal/Symbolic/Contract	temporal logic (LTL) [34], transition systems

Each representation was classified along eight dimensions (abstraction level, V-model phase, underlying mechanism, representation language, formality, testing paradigm, representation object, and usage context) and the 30 representations group into six broad paradigms, summarized in Table I. A single running example, Requirement C of the case-study SRS (§9.2.1, `The support structure for communicating commands must be reliable, with a uniform ACK/NAK protocol adopted across all systems. Timeouts must be supported at approximately 500 msec.`), was instantiated across all 30 representations as a worked catalog, so that what "the same requirement, a different representation" actually looks like in practice is concrete rather than asserted.

A. Worked Example

Table II shows Requirement C instantiated in three representations spanning three different paradigms, to make the taxonomy's central claim concrete: each representation foregrounds a different aspect of the same requirement. The Gherkin scenario (Scenario/Behavioral) foregrounds the observable sequence of events; the finite-state-machine table (Stateful/Transitions) foregrounds the states the subsystem can be in and which inputs move it between them; the decision table (Logical/Combinatorial) foregrounds the combinations of conditions (command sent, response type, elapsed time) that determine each outcome. A model that is only ever asked to produce xUnit-style assertions, regardless of which of these aspects the requirement is actually about, is being asked to flatten this distinction, which is precisely what Gate 1 is designed to catch.

TABLE II
REQUIREMENT C (SRS §9.2.1) IN THREE REPRESENTATIONS

Gherkin	`Scenario: 500ms Watchdog Timeout`
	`Given the subsystem is IDLE`
	`When the workstation sends SELF_TEST`
	`And no response arrives within 500ms`
	`Then status becomes TIMEOUT_ERROR`

FSM table	`(Idle, send_command) ->`
	`(AwaitingResponse, StartTimer)`
	`(AwaitingResponse, recv_ACK) -> (Idle,`
	`StopTimer)`
	`(AwaitingResponse, timer_expired) ->`
	`(TimeoutState, LogTimeout)`

Decision table	`CommandSent=Y, Response=NAK,`
	`Elapsed<500ms=Y → LogProtocolError`
	`CommandSent=Y, Response=none,`
	`Elapsed<500ms=N → RaiseTimeoutException`

TABLE III
THE THREE GATES AT A GLANCE

Gate	Threshold	What it checks
Gate 1: RSS	≥ 0.60	Is the chosen test-case representation actually suited to this requirement's kind of behaviour?
Gate 2: SFV	≥ 0.60	Is the generated suite syntactically well formed, both as canonical JSON and in its chosen notation?
Gate 3: FSA + M_g	≥ 0.50	Does the suite cover the right clauses, including negative and boundary cases, with precise oracles and values grounded in the requirement text?

IV. THE THREE-GATE FRAMEWORK

Every generated suite passes through three gates in sequence, each with a fixed, pre-registered threshold; thresholds below are exactly what the pipeline implementation enforces, not an idealized or aspirational version of the design. Table III summarizes the three gates before the formulas and thresholds are given in full.

Gate 1: Representational Suitability (RSS). Before any test is generated, the model must choose, or be offered, a representation for the requirement at hand. RSS scores that choice heuristically:

$$\mathrm{RSS} = 0.40\,M_1 + 0.30\,M_2 + 0.15\,M_3 + 0.15\,M_4, \qquad (1)$$

where M_1 is concern alignment (does the representation match the kind of property the requirement describes), M_2 is abstraction-level alignment, M_3 penalizes unnecessary representational complexity (Occam's razor), and M_4 scores whether the representation is one an LLM can plausibly generate correctly. The weights $\{0.40, 0.30, 0.15, 0.15\}$ were set by expert elicitation during design: M_1 receives the largest share because choosing the wrong concern class (e.g., a state representation for a pure business rule) is a structural mismatch that higher-weighted metrics cannot compensate for; M_2 addresses the same issue one level of abstraction up. M_3 and M_4 are secondary correctives. To check sensitivity to these weights, we varied each by ± 0.05 (rescaling the others proportionally) for all 59 units (754 unit-level perturbations; `tools/sensitivity.py`). The top-ranked paradigm changed in 2.7% of cases. The full paradigm ordering changed in 33% of cases, with mean Kendall's [48] $\tau = 0.94$ against the baseline ordering, so the top choice is robust but the order of lower-ranked paradigms is not.

TABLE IV
THE SIX METRICS ADDED UNDER THE EXTENDED RSS FORM (M_5-M_{10})

Metric	What it scores
M_5 Oracle Executability	Can a test runner execute the suite's assertions automatically, without adapter code or a human in the loop.
M_6 Traceability Density	How directly each generated test maps back to one specific requirement clause.
M_7 Cognitive Auditability	Whether a human reviewer can read a test and spot a hallucinated value at a glance.
M_8 Change Fragility	Whether a small requirement edit needs only a localized change to the suite, rather than breaking the whole model.
M_9 Mutation Support	Whether the representation supports automated fault injection [46], [47], so the suite's ability to catch real bugs can itself be tested.
M_{10} Standards and Compliance Fit	Whether the representation is an officially recognized formal model for the applicable safety-certification standard (e.g. ISO 26262, DO-178C); capped at 0.65 outside safety-critical systems.

An extended ten-metric form adds six further metrics, M_5 to M_{10} (Table IV), reweighted under six system-profile-specific weight sets (standard, safety-critical, rapid-prototyping, data-pipeline, distributed-concurrent, interaction-heavy) for domain-specific weighting. Each requirement's system profile is inferred automatically from its text; in the benchmark, 249 of 496 suites were scored under the standard profile and the remaining 247 split across four of the other five (interaction-heavy 152, data-pipeline 47, safety-critical 40, distributed-concurrent 8; rapid-prototyping was never selected), so the extended metrics are not a dormant option but an active part of roughly half the reported results. **Threshold: RSS** ≥ 0.60, with up to three re-picks before the pipeline falls back to a default representation.

Gate 2: Syntactic Form Validity (SFV). Once a suite is generated, SFV checks two things separately and blends them evenly:

$$\mathrm{SFV} = 0.5\,S_{\text{schema}} + 0.5\,S_{\text{repr}}. \qquad (2)$$

S_{schema} is the fraction of canonical test cases that validate against the pipeline's JSON schema, meaning every required field is present, case ids are unique, and every `req_id` the case cites traces to a requirement actually present in the unit. S_{repr} is a family-specific syntax check of the representation artifact itself, routed by the suite's *actual* representation group rather than one generic parser applied to everything. Gherkin suites are parsed with the official Gherkin grammar [36] (0.0 on a parse error, 0.4 if it parses but declares no `Scenario`,

TABLE V
GATE 2 REPRESENTATION-SCORE THRESHOLD BY PARADIGM GROUP

Paradigm group (Table I)	S_{repr} **threshold**
Scenario/Behavioral	≥ 0.60
Stateful/Transitions	≥ 0.70
Concurrency/Interaction	≥ 0.60
Logical/Combinatorial	≥ 0.60
Programmatic/Code-level	≥ 1.00
Formal/Symbolic/Contract	≥ 0.70

0.8-1.0 depending on step density); Programmatic/Code-level suites are checked with Python's `ast` parser plus the presence of test functions and assertions; Stateful/Transitions suites are scored on five structural signals (declared states, an initial state, at least two transitions, events/guards, and balanced brackets); Scenario/Behavioral suites that are not Gherkin, such as use cases and user stories, are scored on four signals (a named actor, a main flow, at least one alternative or exception flow, and numbered steps); Concurrency/Interaction, Logical/Combinatorial, and Formal/Symbolic/Contract suites each get an analogous family-specific structural check (arrow and participant markers for sequence diagrams, consistent column counts and separator rows for decision tables, relational operators and matrix-like rows for symbolic notations). A suite passes Gate 2 only if $S_{\text{schema}} = 1.0$ *and* S_{repr} separately clears a per-group threshold (Table V); a perfect schema score cannot compensate for a failing representation score, or vice versa. **Threshold: SFV** ≥ 0.60 by default, tightened to ≥ 0.70 for Stateful/Transitions and Formal/Symbolic/Contract suites, and to 1.00 for Programmatic/Code-level suites, since a syntax error there means the suite does not run at all.

Gate 3: Functional Semantic Adequacy with Groundedness (FSA+M_g). Suites that clear Gates 1 and 2 are reviewed by a 20B open-weight judge model along four adequacy dimensions plus groundedness:

$$\text{FSA} = w_c C_{\text{clause}} + w_n C_{\text{neg}} + w_o A_{\text{oracle}} + w_b C_{\text{bound}} + w_g M_g, \tag{3}$$

with standard-profile weights $w_c = 0.30$ (clause coverage), $w_n = 0.20$ (negative-path coverage), $w_o = 0.15$ (oracle assertiveness), $w_b = 0.20$ (boundary-case coverage), $w_g = 0.15$ (groundedness), and six system profiles analogous to Gate 1. For the standard profile, each FSA component maps to ISO/IEC/IEEE 29119-4 [3]. Clause coverage reflects requirements-based testing [100], whose coverage items are atomic requirements (§5.2.12); it receives the highest weight because an uncovered clause is an untested aspect of the requirement. Negative and boundary coverage receive equal secondary weights, since requirements-based tests are typically positive and negative cases require complementary techniques such as boundary value analysis (§5.2.12.3, §5.2.3). Oracle assertiveness and groundedness assess expected results [96], which the standard requires to be derived from the test basis (§5.1) [4]; a value not traceable to the requirement violates this. The specific weights, in all profiles, are our design choices; as Section VII shows, a low groundedness weight lets ungrounded suites pass, motivating a separate threshold. As with RSS, we varied each FSA weight by ± 0.05 (others rescaled proportionally; `tools/sensitivity.py`). Suite-level pass/fail at FSA ≥ 0.50 changed in 3.6% of decisions (111 of 3,110). Rankings were not fully stable: the paradigm ordering changed in 6 of 10 perturbations, but only through swaps of adjacent paradigms (Kendall's $\tau \geq 0.867$), and the best and worst paradigms were never displaced by more than one rank. Groundedness itself is

$$M_g = 1 - \frac{H}{A}, \tag{4}$$

the fraction of asserted concrete values (A) that are *not* traceable to the source requirement text (H = hallucinated [60]). **Threshold: FSA** ≥ 0.50**.** Suites that fail Gate 3 are fed back for up to three closed-loop repair attempts [61] before being recorded as a final failure. The full pipeline is shown in Fig. 1.

Threshold rationale and design-document discrepancy. The thresholds RSS ≥ 0.60, SFV ≥ 0.60, FSA ≥ 0.50 were chosen conservatively relative to the original design documents, which specified SFV ≥ 0.80 and FSA ≥ 0.75. The thresholds were relaxed so that enough suites reach Gate 3 to study the judge's behaviour; under the implemented thresholds, 311 of 496 suites (63%) reach Gate 3. The implemented thresholds therefore reflect a deliberate trade-off: enough suites reach Gate 3 for comparative analysis at the cost of a lower quality bar. Under the stricter design-document thresholds, only 106 of the 311 judged suites would pass Gate 3 (vs. 214 actual) and the benchmark findings would rest on a substantially smaller sample. We flag this discrepancy explicitly because a methods section that quietly followed the more demanding, unimplemented thresholds would misrepresent the very results it reports, an error of exactly the kind this paper argues the field should stop making silently. All results in this paper reflect the implemented, looser thresholds.

A. Prompt Design

Every prompt used in the pipeline is a plain text template with explicit placeholders, stored and versioned separately from the pipeline code, so that what a model was actually asked is always directly inspectable rather than reconstructed from code. Table VI gives the abbreviated form of the three prompts most central to this paper's argument: the representation choice prompt (Gate 1), the generation prompt (Stage S3), and the judge prompt (Gate 3). Full prompt text is included in the project repository.

The judge prompt (`fsa_judge.txt`) is the one most relevant to Section VII: it instructs the judge to decide, value by value, whether a concrete number, name, or state asserted in the suite is actually entailed by the requirement text, which is exactly the groundedness check later found to disagree so often with what execution against real code shows.

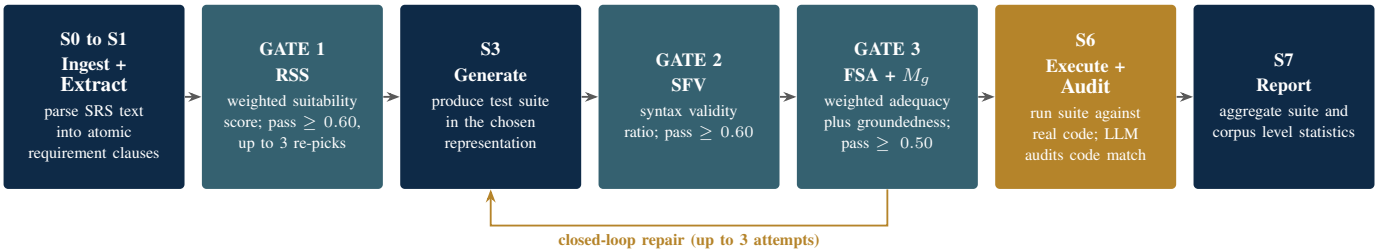


Fig. 1. The requirement-to-test pipeline: extraction, the three gates in sequence with their exact formulas and thresholds as implemented, closed-loop repair on Gate-3 failure, then execution/audit against the real implementation.

TABLE VI
ABBREVIATED PROMPT TEMPLATES FOR THE STAGES CENTRAL TO THIS PAPER (PLACEHOLDERS SHOWN AS {{FIELD}})

Prompt	Used at	Abbreviated template
`select_reps.txt`	Gate 1 (representation choice)	`"Choose the test-case representations best suited to the requirements below. REQUIREMENTS: {{REQS}} DEPENDENCIES: {{DEPS}} AVAILABLE REPRESENTATIONS: {{CATALOG}} Pick the {{K}} most suitable, judged by concern alignment, abstraction level, simplicity, and LLM feasibility. Respond with ONLY a JSON object: {"selected": [{"representation": ..., "covers": [...], "reason": ...}]}"`
`generate.txt`	Stage S3 (test generation)	`"Write a complete test suite for the requirements below in the representation {{REP}}. REQUIREMENTS: {{REQS}} DEPENDENCIES (context only): {{DEPS}} SUFFICIENCY CRITERIA: {{CHECKLIST}} The suite will be scored on: syntactic validity; clause coverage; negative and boundary paths; oracle assertiveness; groundedness (every concrete value must come from the requirement text, never invented). Output exactly two fenced blocks: the artifact in {{REP}} notation, and the same cases as canonical JSON."`
`fsa_judge.txt`	Gate 3 (semantic adequacy judge)	`"You are an independent test-quality evaluator. You did NOT write the suite below. Score it strictly against the SRS requirements, and check every concrete value for hallucination. REQUIREMENTS (ground truth): {{REQS}} ARTIFACT: {{ARTIFACT}} CANONICAL CASES: {{TESTCASES}} Score clause_coverage, negative_coverage, oracle_assertiveness, boundary_coverage in [0,1]; for every concrete element asserted, decide "grounded": true/false by entailment against the SRS text. Respond with ONLY a JSON object: {"clause_coverage": ..., ..., "concrete_elements": [{"value": ..., "grounded": ...}], "weaknesses": [...]}"`

V. EXPERIMENTAL SETUP

A. Model Selection

Candidate models were narrowed from an initial pool of 65 through a four-stage funnel: licensing/recency screening [62], [63] (65→31→29), hardware feasibility by VRAM and context length (29→22), capability and benchmark screening (22→16), and a final state-of-the-art (SOTA)/diversity pass (16→14), yielding 12 open-weight models plus two closed-source controls (GPT-4o, Claude 3.5 Sonnet) intended as reference points.

Of the 12 open-weight models, **10 completed the full benchmark**: Llama-3.1-8B [64], Qwen2.5-Coder-7B, Qwen2.5-Coder-32B [65], Qwen3-30B-A3B [66], Gemma-3-27B [67], CodeGemma-7B [68], DeepSeek-Coder-V2-Lite [69], Granite-4.1-8B [70], Phi-3.5-mini, and Phi-3.5-MoE [71]. Phi-4-14B [72] failed Gate 1 on every attempted unit (server errors mid-generation) and is excluded from the comparative results; Llama-3.3-70B and the two closed-source controls were not executed, for compute and cost reasons. We also note that Gemma-3-27B and DeepSeek-Coder-V2-

Lite completed only a small fraction of the full unit set (7 and 6 suites respectively, against 58 to 61 for the other eight models); their aggregate statistics below should be read with correspondingly lower confidence.

B. Case Studies

Three SRS/implementation pairs were used, each with its generated tests later compared against its real codebase (Section VI):

- **doorstop**: the requirements-management tool Doorstop [73], with its own Python implementation. 13 requirements extracted; time-budgeting restricted generation to 2 units, each run against 2 representations (4 suites per model).
- **ventilator**: an open-source ventilator control system with its own SRS and software implementation. 151 requirements extracted; 21 units generated, one representation per unit.
- **virtual_clinic**: a clinic-management web application (Django-based) with a PDF SRS [74]. 251 requirements extracted; 36 units generated, one representation per unit.

Across the three case studies and 10 executed models this yields **496 test suites and 4,058 test cases** (40 doorstop, 170 ventilator, 286 virtual_clinic suites). The same extraction, judge, and executor model was used for every generator, to avoid a model favouring its own family's outputs [23]. This model is `gpt-oss:20b` [75], OpenAI's open-weight 20B model, served through Ollama [76] on our group's GPU server during the experiment window (September–October 2026); it is from a different family than every open-weight subject under test. The judge temperature was fixed at 0 (seed 42) for all calls. Test generation used only the SRS text; the real implementation was opened only at the execution/audit stage (Stage S6), after every suite had been frozen, to prevent code leakage into generation. **Execution pass criterion.** Generated tests were run with `pytest` [77] against the real implementation. A test counts as *passing* only if `pytest` reports it as passed; failed, errored, and skipped tests (including those the executor marked as not implemented or not automatable) are not passes. Of the 315 doorstop tests, 2 passed, 63 failed, 51 errored, and 180 were skipped.

VI. Results I: Model and Representation Comparison

Fig. 2 shows mean FSA per model pooled across all three case studies. We emphasize a reading hazard visible in this figure: mean FSA is computed only over suites that *reached* the judge (i.e., passed Gate 2), so a model with a very low Gate-2 pass rate can still show a high mean FSA on the small number of suites it did get judged on. Phi-3.5-mini, for instance, shows the single highest pooled mean FSA (0.74) in Fig. 2 despite passing Gate 2 on only 9% of its 58 attempted suites and Gate 3 on only 2%; a leaderboard built from FSA alone would rank it first on the strength of essentially one or two judged suites. On the controlled doorstop comparison (Table VII, where every model attempted the same four suites),

TABLE VII
Gate outcomes by model on the controlled doorstop case study (same 4 units attempted by every model; $n = 4$ suites/model). G2 and G3 are Gate 2 (SFV) and Gate 3 (FSA) pass rates; "both" is the share passing both. **Caution:** $n = 4$ is too small for reliable inter-model ranking; these figures are presented for descriptive comparability on an equal-sample basis only, not as definitive performance rankings.

Model	G2 pass	G3 pass	both	FSA
Llama-3.1-8B	100%	100%	100%	0.73
Qwen2.5-Coder-32B	75%	75%	75%	0.67
Gemma-3-27B	100%	100%	100%	0.66
DeepSeek-Coder-V2-Lite	75%	25%	25%	0.63
Granite-4.1-8B	75%	50%	50%	0.57
Qwen2.5-Coder-7B	50%	50%	50%	0.55
Qwen3-30B-A3B	25%	25%	25%	0.53
Phi-3.5-MoE	25%	25%	25%	0.53
CodeGemma-7B	75%	25%	25%	0.47
Phi-3.5-mini	0%	0%	0%	n/a

Llama-3.1-8B scores highest on this sample (FSA 0.73, both-gates 100%) and CodeGemma-7B lowest among models that passed Gate 1 at all (FSA 0.47, both-gates 25%). We caution that $n = 4$ suites per model is insufficient for reliable inter-model ranking; these figures serve as a controlled descriptive comparison rather than a definitive leaderboard.

Two comparisons that a practitioner might expect to predict quality do not (answering RQ1): model size (Spearman [50] $\rho(\text{parameters}, \text{FSA}) = -0.015$, $p = 0.80$, $n = 311$; a trivially small and non-significant correlation) and code specialization (FSA 0.66 for code-specialized models vs. 0.69 for general-purpose; Mann-Whitney [51] $p = 0.141$, not statistically significant); both comparisons pool all three projects.

Representation choice shows a larger and more consistent directional effect than model identity, addressing RQ2. Fig. 3 breaks FSA and both-gates pass rate down by the six paradigms of Section III: Logical/Combinatorial representations (decision tables, etc.) score highest on mean FSA (0.70) and both-gates pass rate (55%), while Stateful/Transitions representations (FSMs, statecharts) score lowest on FSA (0.62) and both-gates pass rate (23%). A Kruskal-Wallis test [52], [53] across the six paradigm groups yields $H(5) = 14.3$, $p = 0.014$, indicating a statistically significant paradigm effect on FSA; pairwise Dunn tests [54] with Bonferroni correction identify the Logical/Combinatorial – Stateful/Transitions pair as the only reliably different pair ($p_{\text{adj}} = 0.028$, rank-biserial $r = 0.21$, a small-to-medium effect). Differences between other paradigm pairs are not significant after correction and should not be over-interpreted. At the level of individual representations, the 30 available forms are used very unevenly: five representations (Gherkin, xUnit, Use-Cases, Decision Tables, FSMs) account for the overwhelming majority of all choices models made, with 18 of the 30 catalog representations selected at least once. The 12 unused representations are: UML Sequence Charts (formal), Z-notation, Alloy models, OCL constraints, Simulink/Stateflow, TTCN-3 [49], SDL, MSCs, tabular specifications, fault trees, QFD matrices, and robust-

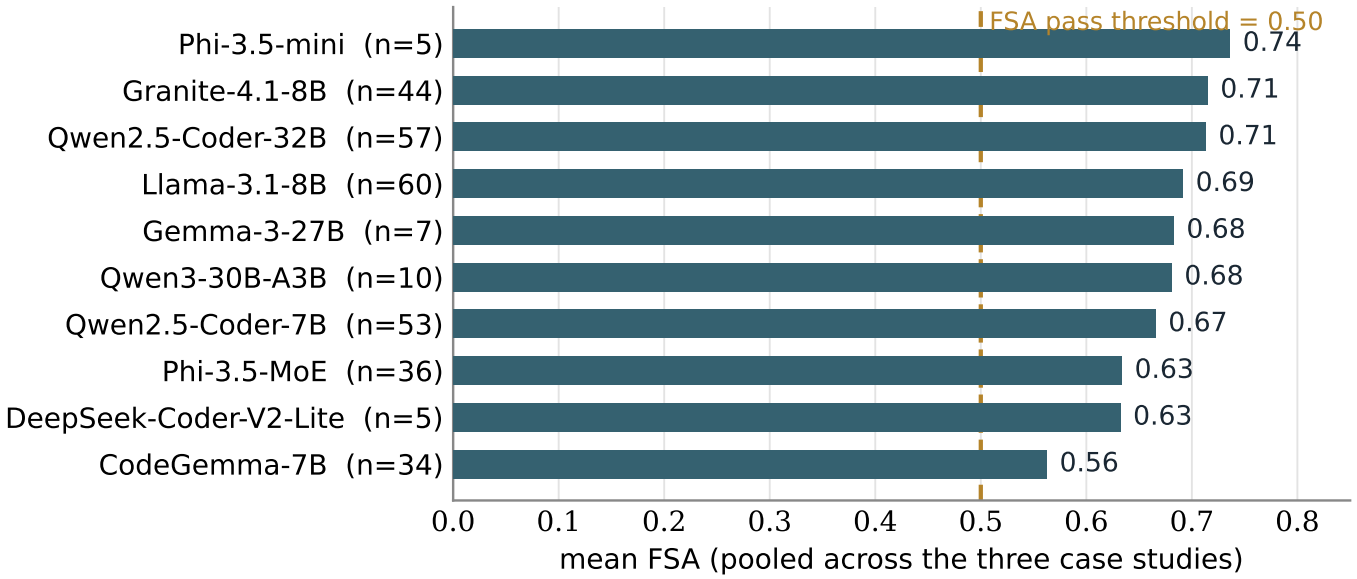


Fig. 2. Mean FSA per model, pooled across all three case studies, with the number of judged suites each mean is computed over. Sample sizes vary sharply by model (6 to 61 judged suites); see text for why this figure alone should not be read as a leaderboard.

ness diagrams. Their non-selection reflects two interacting factors: (i) the three case-study SRS documents contain few requirements whose dominant concern class (formal contracts, safety-critical fault analysis) would trigger the RSS heuristic to rank these forms first; and (ii) informal inspection suggests LLMs have lower confidence generating these notations, consistent with M_4 (LLM feasibility) receiving lower scores for them during Gate 1. This clustering onto familiar notations is itself a finding: the RSS gate's re-pick mechanism and fallback do reduce the bias but do not eliminate it, and 21% Gate-1 agreement with the independent heuristic (Section VII) confirms the models' tendency to gravitate toward a small default set. Strikingly, a model's own Gate-1 choice agrees with the independent RSS heuristic's top-3 recommendation only 21% of the time on average; the two notions of "suitable representation" mostly disagree, a point we return to in Section VII.

Table VIII breaks this down to the level of individual representations with at least 15 suites generated in that form. GUI Event Graphs post the highest both-gates pass rate (64%, $n = 28$) despite a mid-pack FSA; Decision Tables and Gherkin, the two most heavily-used representations, sit in the middle; UML Testing Profile and Use-Cases & User Stories, despite very high syntactic validity (SFV 0.92 and 0.86), both-gates pass at only 27% to 28%, since most of their failures occur at the semantic Gate 3 rather than the syntactic Gate 2, a reminder that a well-formed suite is not automatically an adequate one.

Closed-loop repair is effective where invoked: 207 of the 311 judged suites entered at least one repair attempt. Their mean pre-repair FSA was 0.53 (SD 0.24); after up to three repair iterations, mean FSA rose to 0.66 (SD 0.17), a mean gain of $+0.131$. A paired Wilcoxon signed-rank test [55] on the 207 suites confirms the gain is statistically significant ($W = 314$, $p < 0.001$). Of the 207 repaired suites, 110 (53%) ultimately passed Gate 3; the remaining 97 failed all three attempts and are recorded as final Gate-3 failures.

TABLE VIII
GATE OUTCOMES BY REPRESENTATION ($n \geq 15$ SUITES; n = SUITES GENERATED IN THAT REPRESENTATION; "BOTH-PASS" = SHARE PASSING BOTH GATE 2 AND GATE 3)

Representation	n	SFV	both-pass	FSA
GUI Event Graphs	28	0.96	64%	0.74
Decision Tables	44	0.77	55%	0.70
Gherkin (BDD)	207	0.82	54%	0.67
Sequence Diagrams	18	0.85	44%	0.67
Finite State Machines	40	0.71	28%	0.66
Use-Cases & User Stories	39	0.86	28%	0.62
xUnit Test Cases	79	0.61	29%	0.68
UML Testing Profile (UTP)	15	0.92	27%	0.58

VII. RESULTS II: VALIDITY CHECKS

Section VI reports what the three-gate pipeline says about itself. This section checks those statements against evidence the pipeline did not produce: real test execution, a code-grounded audit, and an independent heuristic. All three checks were built into this study specifically because we did not want to publish a benchmark leaderboard without first asking how much the leaderboard's underlying scores can be trusted.

Execution agreement. On doorstop, the one case study where generated tests were actually executed against the real implementation rather than only audited, the picture is stark: across the ten executed models, **only 2 of 315 generated tests passed** (63 failed, 51 errored, and 180 were skipped, mostly as not implemented). Eight of ten models produced zero passing

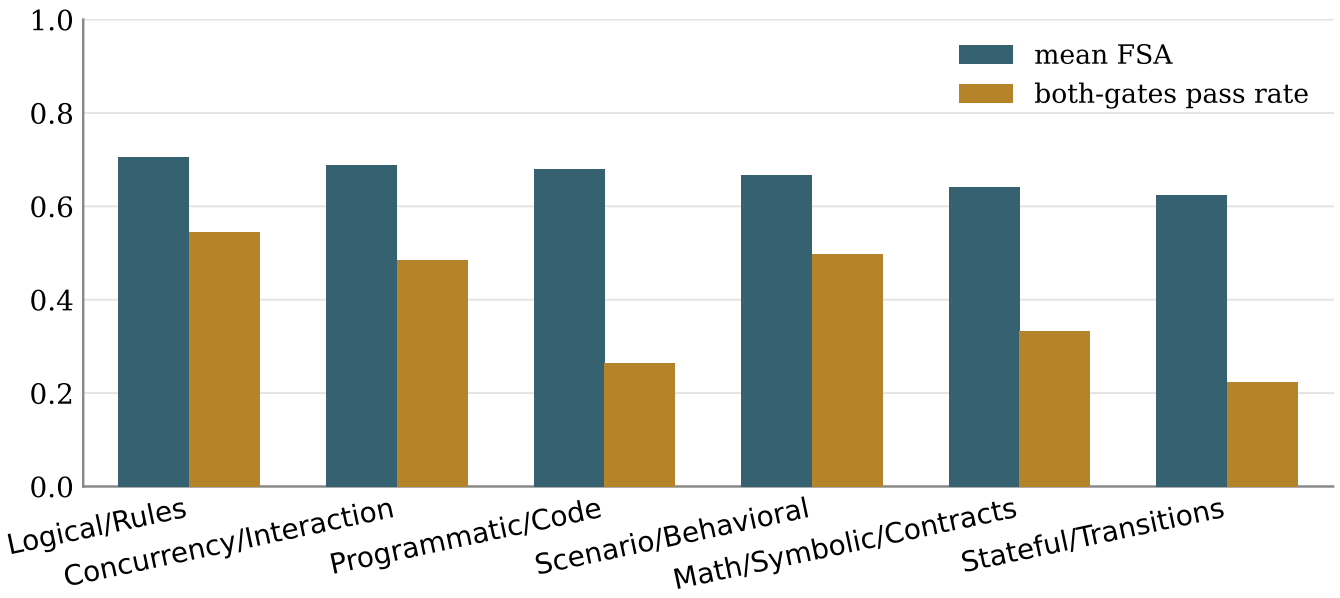


Fig. 3. Mean FSA and both-gates pass rate by representation paradigm, pooled across all models and case studies.

tests out of 10 to 49 attempts each; the only passes were one each for Qwen2.5-Coder-7B (1 of 25) and Qwen2.5-Coder-32B (1 of 33). Gate-passing and real test-execution success are, on this evidence, close to unrelated for this case study: a suite that clears Gates 1 through 3 is not meaningfully more likely to run successfully against the real system than one that does not. The one weak positive signal is a moderate correlation between FSA and execution pass rate across the suites for which both are available ($\rho = 0.433$, $n = 15$, $p = 0.11$, not significant), present but on a small sample, and far from strong enough to license treating a high FSA score as evidence a suite will execute correctly.

Where the execution failures come from. The pytest files actually run in this check are not the generator's native-notation artifact; they are written by the same executor model (`gpt-oss-20b`) from the canonical JSON test case plus a retrieved, necessarily incomplete set of source excerpts (Section V), a translation step with its own failure mode, separate from Gate 3's semantic judgment. Of the 315 cases in the full 10-model doorstop run, 180 (57%) never reached a pass/fail outcome at all: the same executor model marked 136 `not_implemented` and 44 `not_automatable` before running anything, leaving 65 cases actually executed, of which 2 passed, a 3% pass rate among the cases the pipeline itself judged executable. Of the 114 cases that did fail or error, the executor's own three-way triage attributes 107 to `test_bug` (the generated/translated test is wrong) and only 7 to `code_mismatch` (the implementation disagrees with the requirement); the single most common failure signature, 49 of the 114 (43%), is a wrong-arity call into doorstop's own API, for example `Document.new()` invoked without its required `root` and `prefix` arguments. By the pipeline's own account, most observed failures sit in test design or in this translation step, not in doorstop's implementation; the near-zero pass rate reported above should be read accordingly, as evidence against the generated tests and their translation into runnable code, not primarily as a statement about the subject system's correctness. This does not rescue gate-passing as a signal, a suite that clears all three gates is still no more likely to execute than one that does not, but it does change what the failure is evidence of. We flag this triage itself as a single, unverified LLM judgment, for the same reason the audit's 18% agreement rate below counsels caution about any single judgment in this pipeline. Manual inspection of the two passing cases themselves adds a further caveat: both assertions are weak. One checks only that `doorstop.cli` and `doorstop.core` exist as attributes, never exercising the "lightweight installation" requirement it targets; the other wraps the call in `pytest.raises(Exception)`, which passes on any exception at all rather than checking that the specific, informative error message the requirement calls for is shown. The already near-zero positive signal should be read with this in mind.

Audit agreement. Because full execution was only feasible for doorstop (ventilator and virtual_clinic lack an automatable test harness in this study), the other two case studies instead rely on an LLM-judge *code audit* [57], [58]: the same judge model is shown retrieved code excerpts and asked, for each test case, to return one of four verdicts describing whether the code's behaviour matches the test's expectation: `satisfied`, the code does what the test expects; `violated`, the code implements the feature but disagrees with the expectation; `not_implemented`, the feature is absent; or `unclear`, the retrieved excerpt is insufficient to tell. (On doorstop, actually running a test and seeing it fail triggers a separate, three-way triage of *why* it failed, by the same judge model:

`code_mismatch`, the implementation disagrees with the requirement; `test_bug`, the generated test itself is wrong; or `spec_ambiguous`, the requirement does not clearly settle the question.) On the doorstop test cases that were both executed (passed or failed) and given a satisfied or violated audit verdict ($n = 11$), **the audit agrees with real execution on only 2 of 11, i.e. 18%** (95% Wilson [56] CI: 5%–48%). We flag the small sample prominently: a one-sample proportion test at $n = 11$ has very low power, and the true agreement rate could plausibly be as high as 52% or as low as 2%. The directional conclusion, that the LLM audit is an unreliable execution proxy, is supported by the data: all 11 of these tests failed on execution, yet the audit judged 9 of them `satisfied`, consistent with the judge's tendency toward optimistic correctness assessment noted in [19], [22]. We nonetheless recommend that future work replicate this check on a larger doorstop sample before treating 18% as a precise point estimate. This is the single most important caveat on every audit-based result reported for ventilator and virtual_clinic elsewhere in this paper and in the underlying analysis report: audit mode should be read as a weak, frequently-wrong proxy for ground truth, not a substitute for execution. Separately, across all three case studies, the audit flagged 192 individual test cases as contradicting the real implementation, combining an audit `violated` verdict with, on the executed doorstop suites, an execution-triage `code_mismatch` verdict; given the 18% agreement rate just reported, this combined count should be treated as a rough signal requiring manual confirmation, not a verified defect count.

Representation-choice agreement. As noted in Section VI, a model's own Gate-1 pick of representation agrees with the independent RSS heuristic's top-3 recommendation only 21% of the time on average. Gate 1 is designed to make representation choice accountable to an external standard precisely because LLMs left to choose freely cluster onto a handful of familiar notations (Section VI); the low agreement rate shows that accountability mechanism and the models' own judgment are, in practice, measuring different things, and a high Gate-1 pass rate should not be read as confirmation that the chosen representation was actually well suited to the requirement.

Groundedness as a systematically missed failure mode (RQ4). 243 of 311 judged suites (78%) assert at least one concrete value (a timeout, a count, a threshold) that cannot be traced to the source requirement text [59] (mean 12.4 hallucinated values per affected suite; mean $M_g = 0.46$ across all judged suites). Because groundedness is one of five weighted FSA components ($w_g = 0.15$), a suite can clear the FSA ≥ 0.50 threshold while being substantially ungrounded, as long as its other four components score well. We found 112 of 214 Gate-3-passing suites with $M_g < 0.5$, suites the pipeline reports as adequate despite asserting more hallucinated values than grounded ones.

Circularity caveat. The groundedness scores reported above, and therefore the 78% figure, are themselves produced by the same LLM judge whose reliability is called into question by the execution check. A judge that over-optimistically reports suites as `grounded: true` would cause us to *undercount* hallucinated suites; the true proportion of suites with at least one ungrounded value is likely no lower than 78% and may be substantially higher. This circularity does not invalidate the groundedness finding, but it means the 78% figure should be treated as a lower bound rather than a precise estimate. This is precisely why we recommend promoting groundedness to an independently thresholded gate verified by a secondary mechanism (e.g., automated lexical entailment against the SRS text), rather than relying solely on the same judge that produces FSA.

Against the real implementation, directly. Even setting execution and audit reliability aside, the *best-case* agreement between generated tests and real requirement-level behaviour, measured as the share of SRS requirements whose tests reach actually-implemented behaviour, is 54% (DeepSeek-Coder-V2-Lite, doorstop); the ceiling across every model×project pair in this study is just over half.

Summary of Results I and II together. The pipeline reports both-gates pass rates of 25% to 100% depending on model and case study (Table VII), and a representation-paradigm effect that is statistically significant if small (Section VI). Set against real execution, real code behaviour, and an independent heuristic, each of those signals shrinks: 18% audit-execution agreement, 2 passing tests out of 315 attempted, 21% representation-choice agreement, and roughly three quarters of judged suites carrying at least one hallucinated value. A three-gate, LLM-judged pipeline is not nothing: it is more structured and more falsifiable than a single similarity score, but on this evidence it is not yet a substitute for execution-grounded validation, and should not be reported as one.

VIII. Threats to Validity

Single run per configuration. Every model×unit× representation combination was run once at temperature 0; the underlying analysis explicitly notes that differences of a few FSA points between models are within expected noise and should not be over-interpreted [53].

Single judge model. All semantic adequacy, groundedness, and code audit judgments were made by one model (`gpt-oss-20b`). We chose a model from a different family than every subject under test to reduce self-preference bias, a judge limitation documented in prior LLM-as-judge work [19], [23], but we did not cross-validate its judgments against a second judge model or human raters [99] beyond the audit-vs-execution check in Section VII, which itself shows that judge's reliability is limited.

Execution harness is itself LLM-written. The runnable pytest files checked in Section VII are not hand-written adapters; they are generated by the same executor model from the canonical test case and a retrieved, necessarily incomplete set of source excerpts (Section V). An execution failure can therefore originate in Gate 1 through 3 test design, in this translation step, or in the subject implementation, and this

study cannot fully separate the three from execution results alone. The pipeline's own triage (Section VII) attributes most doorstop failures to the test/translation side rather than the implementation, but that triage is itself a single, unverified LLM judgment, not a result this paper independently confirmed.

Uneven execution coverage. Real test execution was only feasible for doorstop; ventilator and virtual_clinic results rely on the audit proxy, which Section VII shows agrees with execution only 18% of the time on the one case study where both are available. Conclusions drawn from ventilator/virtual_clinic audit data alone should be read as provisional.

Partial model coverage. Gemma-3-27B and DeepSeek-Coder-V2-Lite completed only 6 to 7 suites each, against 58 to 61 for the other eight executed models; Phi-4-14B failed to pass Gate 1 on any attempted unit; Llama-3.3-70B and both closed-source controls (GPT-4o, Claude 3.5 Sonnet) were not executed. The 10-model comparison in this paper is therefore narrower than the 14-model selection funnel that produced it, and entirely open-weight.

Threshold sensitivity. The implemented gate thresholds (RSS ≥ 0.60, SFV ≥ 0.60, FSA ≥ 0.50) are looser than the design-document values (SFV ≥ 0.80, FSA ≥ 0.75). Under the stricter thresholds, 106 of the 311 judged suites would pass Gate 3 (vs. 214 actual); all 311 already have SFV above 0.80. The paradigm ranking of Section VI was computed under the implemented thresholds only and was not recomputed under the stricter ones. The absolute pass rates in Table VII would drop materially under the stricter thresholds and should be interpreted as lower-bound quality estimates specific to the implemented configuration.

Three case studies. doorstop, ventilator, and virtual_clinic differ in size and domain but are three systems; generalizing findings here to SRS documents of very different style, domain, or scale (e.g., safety-critical avionics specifications) should be done cautiously [98].

IX. Discussion

The clearest actionable finding from Section VI is that *which representation a requirement is tested with matters more than which model generates the test*, a result consistent with the motivating premise of this paper's own taxonomy (Section III) [29], and one that argues for representation-aware tooling over simply swapping in a larger or more code-specialized model.

The more consequential finding, from Section VII, is that this and plausibly other LLM-judge-based evaluation pipelines should not be trusted to report correctness without an execution-grounded check built in and reported alongside the gate scores. We did not find this gap by assuming our framework was flawed; we found it by deliberately building the cross-checks (execution comparison, code audit, independent heuristic) that a pipeline reporting only its own gate scores would never surface on its own. We think this is the more general lesson: any LLM-as-judge evaluation pipeline for generated tests should report at least one such cross-check, not only its own internal pass rates, and groundedness in particular is well-suited to being promoted from a weighted component of a blended score to an independently thresholded gate, given how cleanly it can hide inside a passing FSA score (Section VII).

X. Planned Changes

Table IX collects, in one place, every validity gap this paper measured and the specific change we plan to make in response to it. Each row names the evidence (already reported in Section VII) rather than repeating it, so this table should be read as a punch list rather than a new set of claims.

A. Representation Mapping Rules

The 21% agreement between a model's free Gate-1 choice and the independent RSS heuristic (Section VII) shows the two views of "suitable representation" are, at present, only loosely connected. We plan to formalize explicit mapping rules from requirement dependency type (state, interaction, data, concurrency, as used in the RSS heuristic itself) to a short ranked list of representations, so that representation selection is governed by a documented, inspectable rule set rather than left to either an LLM's free choice or a keyword heuristic alone.

B. Full Execution Coverage

Real execution in this study was only feasible for doorstop; ventilator and virtual_clinic results rely on the weaker audit proxy, which Section VII shows agrees with execution only 18% of the time on the one case study where both are available. The next iteration of this work builds an automatable execution harness for both remaining case studies, so that the 18% audit-execution agreement figure can either be confirmed as a general property of this evaluation approach or shown to be specific to doorstop.

C. System-Level Test Generation

Every suite reported in this paper covers exactly one requirement unit, up to 8 requirements drawn from a single SRS section, in isolation. Nothing in the current framework exercises a flow that spans multiple units, a dependency between two separately specified requirements, an environment or component failure, or a quality attribute (latency, throughput, reliability) that only manifests across the running system rather than inside one function. Closing this gap is the most structurally different piece of planned work, since it requires a ground-truth structure the current unit-level canonical schema (Section IV) does not provide: an explicit coverage denominator at the system level, analogous to what traceability already gives at the unit level.

We are evaluating two candidate ground-truth formats for this purpose, shown in Fig. 4 and summarized below.

Candidate 1: a structured, dimension-based schema. Every test is one cell addressed by a composite id, for example `E2E.F01.web-ui.negative.add-duplicate`, decomposing into a flow, an interface, an input class, and a name. Each cell appears in three places: a core record

TABLE IX
EVERY VALIDITY GAP MEASURED IN THIS PAPER, AND THE PLANNED CHANGE ADDRESSING IT

Gap	Evidence (Section VII)	Planned change
Gate-passing does not predict execution success	2 of 315 generated doorstop tests actually pass; $\rho = 0.433$ between FSA and execution pass rate, $n = 15$	Report an execution-grounded pass rate alongside every gate score, not only on doorstop (see "Full Execution Coverage" below)
Code audit is an unreliable stand-in for execution	Audit agrees with real execution only 18% of the time ($n = 11$)	Replace audit-only results for ventilator and virtual_clinic with real execution, and report audit-execution agreement per project rather than assuming it transfers
Representation choice is not accountable to an external standard	Model's own Gate-1 pick agrees with the independent RSS heuristic's top-3 only 21% of the time	Formalize explicit dependency-type-to-representation mapping rules (see "Representation Mapping Rules" below) rather than leaving the mapping heuristic
Groundedness can hide inside a passing adequacy score	243 of 311 judged suites (78%) assert an ungrounded value; 112 of 214 Gate-3-passing suites have $M_g < 0.5$	Promote groundedness from a weighted FSA component to its own independently thresholded gate
Evaluation stops at a single requirement unit	Every suite in this study covers one unit of up to 8 requirements in isolation; no end-to-end, interaction, failure, or quality-attribute coverage is measured	Extend the framework to system-level testing with an explicit, multi-dimensional coverage denominator (see "System-Level Test Generation" below)
Execution failures cannot be attributed to test design, translation, or implementation	107 of 114 triaged doorstop failures judged `test_bug` vs. 7 `code_mismatch`, by the same single judge model that also wrote the executed code	Separate the translate-to-runnable-code step from judged test design, e.g. deterministic/templated code generation or a second, independent judge for execution triage

(preconditions, steps with an expected result per step, test data), a dimension-specific record (one of D1 end-to-end, D2 interaction between two requirements, D3 environment or component failure, D4 a quality attribute tested below, at, or above its threshold), and an inventory line carrying the requirement references that cell covers. A sync rule, every cell id appears exactly once in the core, once in its dimension, and once in the inventory, makes system-level coverage mechanically checkable the same way unit-level traceability already is in this paper's framework (a pilot inventory built this way already holds 189 cells across the four dimensions).

Candidate 2: an industrial, spreadsheet-style sheet. A simpler alternative closer to common ISO/IEC/IEEE 29119 [4] (the international software testing standard) style test-case documentation: one row per test, with id, name, requirement links, precondition, steps, expected result, postcondition, and use case as columns, plus a separate methodology sheet classifying each row as happy-path, negative, or boundary. This format is easier for a human reviewer to read and already separates expected result from postcondition, but it has no explicit coverage-item inventory or denominator, so system-level coverage cannot be computed from it the way it can from Candidate 1, and concrete test-data values are more often left as vague placeholders.

We plan to pilot Candidate 1 first, since its explicit, checkable denominator extends this paper's existing traceability methodology most directly from the unit level to the system level, and to compare it against Candidate 2 on reviewer readability and on whether the heavier structure measurably improves coverage auditability in practice before committing to either format for the full benchmark.

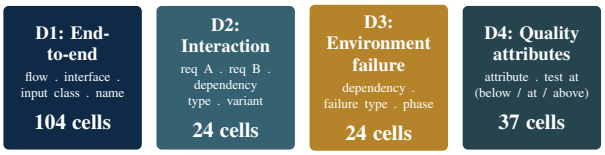
Future Work Beyond This Table

Three further gaps are worth naming even without a dedicated row in Table IX: cross-validating the single judge model against a second judge or human raters; replicating the sensitivity analysis on a larger, independently-sourced unit set; and re-running the 21% representation-choice agreement figure once the mapping rules above are in place, to see whether a documented rule set closes the gap or merely relocates it.

XI. RELATED WORK

Classical and SBST test generation. Search-based software testing (SBST) [78] approaches such as EvoSuite [79] and Randoop [80] evolve test suites against coverage or mutation-based fitness functions without requiring a natural-language source. Rojas et al. [81] study the combination of multiple adequacy criteria in a single SBST objective, motivating our multi-gate design. These approaches produce executable tests but require compilable code stubs and do not consume requirements text; this paper addresses the upstream, requirement-driven generation step.

LLM-based test generation. Tufano et al. [82] establish a Transformer baseline for unit test generation using focal-context embeddings. More recently, Schafer et al. [14] empirically evaluate LLMs for unit test generation across multiple models and find that model specialization and size do not reliably predict test quality, consistent with our RQ1 findings.



Every cell carries a composite id, for example `E2E.F01.web-ui.negative.add-duplicate`, and appears once in a shared **Core** record (preconditions, steps, expected result, test data) and once in a shared **Inventory** line (requirement references, the coverage denominator), the sync rule that makes system-level coverage mechanically checkable.

Fig. 4. The four coverage dimensions of the structured ground-truth candidate for planned system-level testing (Section X-C), with the pilot inventory's cell count per dimension (189 cells total).

Yuan et al. [15] evaluate ChatGPT-generated JUnit tests and introduce execution-based quality checks, finding that many syntactically valid LLM tests fail at runtime; our execution pass rates ($\leq$2/315) are even lower, likely because we target requirement-to-test generation (where no code stub is given) rather than code-to-test generation. Siddiq et al. [16] study LLM-generated test assertion correctness specifically and find substantial hallucination in asserted values, corroborating our groundedness finding. TestGen-LLM [83] augments existing test suites using coverage feedback [84]–[86]; we complement this by studying whether LLM-generated tests are adequate in the first place when generated from requirements. For requirements-to-test specifically, Hossain et al. [87] generate test cases from natural-language requirements using LLMs and evaluate with BLEU, precisely the approach this paper argues should be augmented with execution-grounded checks.

LLM-as-judge evaluation. The use of a second LLM to evaluate generated artefacts is surveyed in Gu et al. [21]; our paper is, to our knowledge, the first to measure the measurement-validity gap of an LLM judge specifically in the test-generation context, via comparison against real execution. Reflexion [61] applies LLM self-feedback to agentic tasks; our closed-loop repair is structurally similar but targeted at test-suite adequacy scores rather than task completion signals. SWE-bench [88] and SWE-agent [89] share this paper's concern with evaluating LLM outputs against executable ground truth rather than static references, in the issue-resolution setting. Deng et al. [90] use LLMs as zero-shot fuzzers [91] with execution-grounded evaluation, consistent with the direction we recommend.

Test adequacy metrics. Inozemtseva & Holmes [92] show that statement coverage is a weak proxy for real fault-detection ability [47], [93], a precursor to this paper's argument that gate-pass rates are weak proxies for execution correctness. Our oracle assertiveness component (FSA) is consistent with the test oracle generation literature [94], [95] reviewed by Barr et al. [96].

Model licensing and openness considerations in our model-selection funnel draw on [62], [63]; the open-weight code models are from families documented in [64]–[71], with general LLM background from [97].

XII. Conclusion

We set out to build a careful, multi-gate alternative to text-similarity metrics for evaluating LLM-generated, requirement-derived test suites, and benchmarked it across 10 models and three real SRS/implementation pairs. The resulting comparative findings, that model size and code specialization do not predict adequacy while representation choice does, are useful on their own. But the paper's central result is that the pipeline's own gate-passing, checked against real execution, a code audit, and an independent heuristic, agrees with ground truth far less often than its pass rates alone would suggest. We report this as a finding about the field's current direction, not only about our own pipeline: as LLM-as-judge evaluation becomes the default replacement for text-similarity metrics in test generation research, it needs to be held to the same standard it was introduced to meet, with execution-grounded validity checks reported as a matter of course rather than as an afterthought.

Data and Code Availability

The pipeline, prompts, gate implementation, aggregated results (`suites.csv`, the analysis report), and this paper's own source are published at https://github.com/poojitha376sp/Clean_exp. The planned system-level ground-truth templates discussed in Section X-C are tracked in the same repository under `paper/`.

References

[1] ISO/IEC/IEEE 29148:2018, "Systems and software engineering – Life cycle processes – Requirements engineering," International Organization for Standardization, 2018.
[2] ISO/IEC/IEEE 29119-1:2022, "Software and systems engineering – Software testing – Part 1: General concepts," International Organization for Standardization, 2022.
[3] ISO/IEC/IEEE 29119-4:2021, "Software and systems engineering – Software testing – Part 4: Test techniques," International Organization for Standardization, 2021.
[4] ISO/IEC/IEEE 29119-2:2021, "Software and systems engineering – Software testing – Part 2: Test processes," International Organization for Standardization, 2021.
[5] V. Garousi, M. Bauer, and M. Felderer, "NLP-assisted software testing: A systematic mapping of the literature," Information and Software Technology, vol. 126, 106321, 2020.
[6] C. Wang, F. Pastore, and L. Briand, "Automatic generation of acceptance test cases from use case specifications: an NLP-based approach," IEEE Transactions on Software Engineering, vol. 48, no. 2, pp. 585–616, 2022.

[7] J. Wang et al., "Software testing with large language models: Survey, landscape, and vision," IEEE Transactions on Software Engineering, vol. 50, no. 4, pp. 911–936, 2024.

[8] X. Hou et al., "Large language models for software engineering: A systematic literature review," ACM Transactions on Software Engineering and Methodology, 2024.

[9] A. Fan et al., "Large language models for software engineering: Survey and open problems," in Proc. IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE), 2023.

[10] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a method for automatic evaluation of machine translation," in Proc. 40th Annual Meeting of the Association for Computational Linguistics (ACL), 2002, pp. 311–318.

[11] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in Text Summarization Branches Out, ACL Workshop, 2004, pp. 74–81.

[12] S. Ren et al., "CodeBLEU: a method for automatic evaluation of code synthesis," arXiv preprint arXiv:2009.10297, 2020.

[13] M. Evtikhiev et al., "Out of the BLEU: how should we assess quality of the code generation models?" Journal of Systems and Software, vol. 203, 111741, 2023.

[14] M. Schafer, S. Nadi, A. Eghbali, and F. Tip, "An empirical evaluation of using large language models for automated unit test generation," IEEE Transactions on Software Engineering, vol. 50, no. 1, pp. 85–105, 2023.

[15] Z. Yuan et al., "No more manual tests? Evaluating and improving ChatGPT for unit test generation," arXiv preprint arXiv:2305.04207, 2023.

[16] M. L. Siddiq et al., "An empirical study of using large language models for unit test generation," in Proc. 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), 2023.

[17] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[18] M. Chen et al., "Evaluating large language models trained on code," arXiv preprint arXiv:2107.03374, 2021.

[19] L. Zheng et al., "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[20] Y. Liu et al., "G-Eval: NLG evaluation using GPT-4 with better human alignment," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2023.

[21] J. Gu et al., "A survey on LLM-as-a-judge," arXiv preprint arXiv:2411.15594, 2024.

[22] P. Wang et al., "Large language models are not fair evaluators," in Proc. 62nd Annual Meeting of the Association for Computational Linguistics (ACL), 2024.

[23] A. Panickssery, S. Bowman, and S. Feng, "LLM evaluators recognize and favor their own generations," in Advances in Neural Information Processing Systems (NeurIPS), 2024.

[24] P. Ralph, "Toward methodological guidelines for process theories and taxonomies in software engineering," IEEE Transactions on Software Engineering, vol. 45, no. 7, pp. 712–735, 2018.

[25] C. Kaner and W. P. Bond, "Software engineering metrics: What do they measure and how do we know?" in Proc. 10th International Software Metrics Symposium, 2004.

[26] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Technical Report EBSE-2007-01, Keele University and University of Durham, 2007.

[27] J. Cohen, "A coefficient of agreement for nominal scales," Educational and Psychological Measurement, vol. 20, no. 1, pp. 37–46, 1960.

[28] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," Biometrics, vol. 33, no. 1, pp. 159–174, 1977.

[29] M. Utting, A. Pretschner, and B. Legeard, "A taxonomy of model-based testing approaches," Software Testing, Verification and Reliability, vol. 22, no. 5, pp. 297–312, 2012.

[30] P. Ammann and J. Offutt, Introduction to Software Testing, 2nd ed. Cambridge University Press, 2016.

[31] T. S. Chow, "Testing software design modeled by finite-state machines," IEEE Transactions on Software Engineering, vol. SE-4, no. 3, pp. 178–187, 1978.

[32] D. Lee and M. Yannakakis, "Principles and methods of testing finite state machines – a survey," Proceedings of the IEEE, vol. 84, no. 8, pp. 1090–1123, 1996.

[33] L. C. Briand and Y. Labiche, "A UML-based approach to system testing," Software and Systems Modeling, vol. 1, no. 1, pp. 10–42, 2002.

[34] A. Pnueli, "The temporal logic of programs," in Proc. 18th Annual Symposium on Foundations of Computer Science (FOCS), 1977, pp. 46–57.

[35] G. J. Myers, C. Sandler, and T. Badgett, The Art of Software Testing, 3rd ed. Wiley, 2011.

[36] M. Wynne and A. Hellesoy, The Cucumber Book: Behaviour-Driven Development for Testers and Developers. Pragmatic Bookshelf, 2012.

[37] C. Nebut, F. Fleurey, Y. Le Traon, and J.-M. Jezequel, "Automatic test generation: a use case driven approach," IEEE Transactions on Software Engineering, vol. 32, no. 3, pp. 140–155, 2006.

[38] D. Harel, "Statecharts: A visual formalism for complex systems," Science of Computer Programming, vol. 8, no. 3, pp. 231–274, 1987.

[39] A. M. Memon, M. E. Pollack, and M. L. Soffa, "Hierarchical GUI test case generation using automated planning," IEEE Transactions on Software Engineering, vol. 27, no. 2, pp. 144–155, 2001.

[40] A. Memon, I. Banerjee, and A. Nagarajan, "GUI ripping: Reverse engineering of graphical user interfaces for testing," in Proc. 10th Working Conference on Reverse Engineering, 2003.

[41] T. J. Ostrand and M. J. Balcer, "The category-partition method for specifying and generating fuctional tests," Communications of the ACM, vol. 31, no. 6, pp. 676–686, 1988.

[42] M. Grochtmann and K. Grimm, "Classification trees for partition testing," Software Testing, Verification and Reliability, vol. 3, no. 2, pp. 63–82, 1993.

[43] C. Nie and H. Leung, "A survey of combinatorial testing," ACM Computing Surveys, vol. 43, no. 2, pp. 1–29, 2011.

[44] G. Meszaros, xUnit Test Patterns: Refactoring Test Code. Addison-Wesley, 2007.

[45] P. Baker et al., Model-Driven Testing: Using the UML Testing Profile. Springer, 2008.

[46] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," IEEE Transactions on Software Engineering, vol. 37, no. 5, pp. 649–678, 2011.

[47] R. Just et al., "Are mutants a valid substitute for real faults in software testing?" in Proc. 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE), 2014.

[48] M. G. Kendall, "A new measure of rank correlation," Biometrika, vol. 30, no. 1/2, pp. 81–93, 1938.

[49] J. Grabowski et al., "An introduction to the testing and test control notation (TTCN-3)," Computer Networks, vol. 42, no. 3, pp. 375–403, 2003.

[50] C. Spearman, "The proof and measurement of association between two things," American Journal of Psychology, vol. 15, no. 1, pp. 72–101, 1904.

[51] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," Annals of Mathematical Statistics, vol. 18, no. 1, pp. 50–60, 1947.

[52] W. H. Kruskal and W. A. Wallis, "Use of ranks in one-criterion variance analysis," Journal of the American Statistical Association, vol. 47, no. 260, pp. 583–621, 1952.

[53] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," Software Testing, Verification and Reliability, vol. 24, no. 3, pp. 219–250, 2014.

[54] O. J. Dunn, "Multiple comparisons using rank sums," Technometrics, vol. 6, no. 3, pp. 241–252, 1964.

[55] F. Wilcoxon, "Individual comparisons by ranking methods," Biometrics Bulletin, vol. 1, no. 6, pp. 80–83, 1945.

[56] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," Journal of the American Statistical Association, vol. 22, no. 158, pp. 209–212, 1927.

[57] T. Y. Zhuo, "ICE-Score: Instructing large language models to evaluate code," in Findings of the Association for Computational Linguistics: EACL, 2024.

[58] W. Tong and T. Zhang, "CodeJudge: Evaluating code generation with large language models," in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2024.

[59] F. Liu et al., "Exploring and evaluating hallucinations in LLM-powered code generation," arXiv preprint arXiv:2404.00971, 2024.

[60] Z. Ji et al., "Survey of hallucination in natural language generation," ACM Computing Surveys, vol. 55, no. 12, pp. 1–38, 2023.

[61] N. Shinn et al., "Reflexion: Language agents with verbal reinforcement learning," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[62] A. Liesenfeld, A. Lopez, and M. Dingemanse, "Opening up ChatGPT: Tracking openness, transparency, and accountability in instruction-tuned text generators," in Proc. 5th International Conference on Conversational User Interfaces (CUI), 2023.

[63] R. White et al., "Model openness framework: Promoting completeness and openness for reproducibility, transparency, and usability in artificial intelligence," arXiv preprint arXiv:2403.13784, 2024.

[64] A. Grattafiori et al., "The Llama 3 herd of models," arXiv preprint arXiv:2407.21783, 2024.

[65] B. Hui et al., "Qwen2.5-Coder technical report," arXiv preprint arXiv:2409.12186, 2024.

[66] A. Yang et al., "Qwen3 technical report," arXiv preprint arXiv:2505.09388, 2025.

[67] Gemma Team, Google DeepMind, "Gemma 3 technical report," arXiv preprint arXiv:2503.19786, 2025.

[68] CodeGemma Team, Google DeepMind, "CodeGemma: Open code models based on Gemma," arXiv preprint arXiv:2406.11409, 2024.

[69] Q. Zhu et al., "DeepSeek-Coder-V2: Breaking the barrier of closed-source models in code intelligence," arXiv preprint arXiv:2406.11931, 2024.

[70] IBM Granite Team, "Granite 4.0: Hybrid Mamba-2/Transformer language models," IBM Research Technical Report, 2025.

[71] M. Abdin et al., "Phi-3 technical report: A highly capable language model locally on your phone," arXiv preprint arXiv:2404.14219, 2024.

[72] M. Abdin et al., "Phi-4 technical report," arXiv preprint arXiv:2412.08905, 2024.

[73] J. Browning, "Doorstop: Text-based requirements management using version control," open-source software, available: https://github.com/doorstop-dev/doorstop, 2014.

[74] Representative clinic-management system SRS used as a case study in this benchmark (project-internal document, not independently published).

[75] OpenAI, "gpt-oss-120b & gpt-oss-20b model card," arXiv preprint arXiv:2508.10925, 2025.

[76] Ollama Inc., "Ollama: Get up and running with large language models locally," available: https://ollama.com, 2024.

[77] H. Krekel et al., "pytest: helps you write better programs," available: https://pytest.org, 2004–present.

[78] P. McMinn, "Search-based software test data generation: a survey," Software Testing, Verification and Reliability, vol. 14, no. 2, pp. 105–156, 2004.

[79] G. Fraser and A. Arcuri, "EvoSuite: automatic test suite generation for object-oriented software," in Proc. 19th ACM SIGSOFT Symposium and 13th European Conference on Foundations of Software Engineering (ESEC/FSE), 2011, pp. 416–419.

[80] C. Pacheco, S. K. Lahiri, M. D. Ernst, and T. Ball, "Feedback-directed random test generation," in Proc. 29th International Conference on Software Engineering (ICSE), 2007, pp. 75–84.

[81] J. M. Rojas, G. Fraser, and A. Arcuri, "Combining multiple coverage criteria in search-based unit test generation," in Proc. 7th International Symposium on Search-Based Software Engineering (SSBSE), 2015.

[82] M. Tufano et al., "Unit test case generation with transformers and focal context," arXiv preprint arXiv:2009.05617, 2022.

[83] N. Alshahwan et al., "Automated unit test improvement using large language models at Meta," in Proc. 32nd ACM International Conference on the Foundations of Software Engineering (FSE), Industry Track, 2024.

[84] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "CODAMOSA: Escaping coverage plateaus in test generation with pre-trained large language models," in Proc. 45th International Conference on Software Engineering (ICSE), 2023.

[85] J. Pizzorno and E. D. Berger, "CoverUp: Coverage-guided LLM-based test generation," arXiv preprint arXiv:2403.16218, 2024.

[86] Y. Chen et al., "ChatUniTest: A framework for LLM-based test generation," in Companion Proc. 32nd ACM International Conference on the Foundations of Software Engineering (FSE), 2024.

[87] S. B. Hossain, M. Jiang, and V. Filkov, "A study on using large language models for generating test cases from software requirements," preprint, 2024.

[88] C. E. Jimenez et al., "SWE-bench: Can language models resolve real-world GitHub issues?" in Proc. International Conference on Learning Representations (ICLR), 2024.

[89] J. Yang et al., "SWE-agent: Agent-computer interfaces enable automated software engineering," in Advances in Neural Information Processing Systems (NeurIPS), 2024.

[90] Y. Deng et al., "Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models," in Proc. 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA), 2023.

[91] C. S. Xia et al., "Fuzz4All: Universal fuzzing with large language models," in Proc. 46th International Conference on Software Engineering (ICSE), 2024.

[92] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," in Proc. 36th International Conference on Software Engineering (ICSE), 2014, pp. 435–445.

[93] H. Zhu, P. A. V. Hall, and J. H. R. May, "Software unit test coverage and adequacy," ACM Computing Surveys, vol. 29, no. 4, pp. 366–427, 1997.

[94] E. Dinella et al., "TOGA: A neural method for test oracle generation," in Proc. 44th International Conference on Software Engineering (ICSE), 2022.

[95] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring LLM-based general bug reproduction," in Proc. 45th International Conference on Software Engineering (ICSE), 2023.

[96] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," IEEE Transactions on Software Engineering, vol. 41, no. 5, pp. 507–525, 2015.

[97] L. Ouyang et al., "Training language models to follow instructions with human feedback," in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, pp. 27730–27744, 2022.

[98] C. Wohlin et al., Experimentation in Software Engineering. Springer, 2012.

[99] T. Ahmed et al., "On the reliability of human annotation for evaluating code quality," preprint, 2025.

[100] M. W. Whalen, A. Rajan, M. P. E. Heimdahl, and S. P. Miller, "Coverage metrics for requirements-based testing," in Proc. International Symposium on Software Testing and Analysis (ISSTA), 2006, pp. 25–36.